Assignment 1

Yahuan Shi, Armin Puran Youssef, Eckart Cobo-Briesewitz

2025.10.27

A. Calculations

1. Compute the gravity vector

The gravity vector $\mathbf{G}$ for estimating the torque at each joint is:

$$\mathbf{G}(q_1, q_2, q_3) = \begin{bmatrix} g(m_1 r_1 c_1 + m_2(L_1 c_1 + r_2 s_{12}) + m_3(L_1 c_1 + L_2 s_{12} + r_3 s_{123})) \\ g(m_2 r_2 s_{12} + m_3(L_2 s_{12} + r_3 s_{123})) \\ g m_3 r_3 s_{123} \end{bmatrix}$$

where:

$$\begin{aligned} c_1 &= \cos(q_1) \\ s_{12} &= \sin(q_1 + q_2) \\ s_{123} &= \sin(q_1 + q_2 + q_3) \end{aligned}$$

B. Implementations

1.[12 Points] njmoveControl()

```
void njmoveControl(GlobalVariables& gv)
{
  gv.tau = gv.kp * (gv.qd - gv.q);   // P-controller
}
```

2.[10 Points] Tune the controllers

Questions:

(a) What kind of behaviors do you observe with different gains?

- Lower gains: There is less oscillation around the target joint state, but the error between the real joint state and the desired joint state is larger.
- Higher gains: The error between real joint state and the target joint state is lowers but there is more oscillation.

(b) Why are well tuned gains different for each joint?

Each joint is supporting a different amount of weight and can output different torques.

3.[10 Points] Document Behavior

Results are shown in Fig. 1 and Fig. 2.

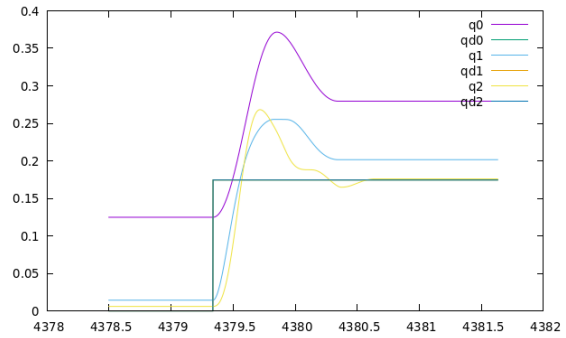

Figure 1: Desired joint states $qd*$ and actual joint states $q*$ for *njmoveControl* with Kp values: $q_1 = 400$, $q_2 = 200$, $q_3 = 25$.

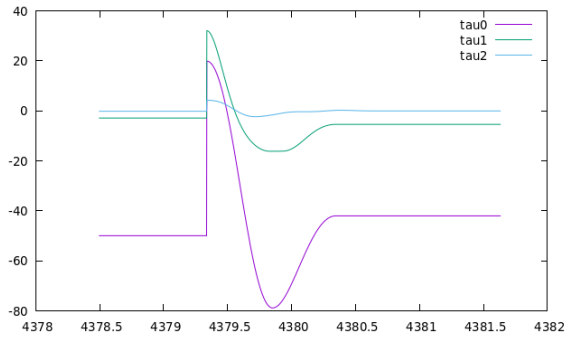

Figure 2: Applied torques $\tau*$ for *njmoveControl* with Kp values: $q_1 = 400$, $q_2 = 200$, $q_3 = 25$.

4.[12 Points] Calculate the gravity vector in PreprocessControl()

```
float r1 = R2;
float r2 = 0.189738;
float r3 = R6;
float l1 = L2;
float l2 = L3;
float l3 = L6;
float m1 = M2;
float m2 = M3 + M4 + M5;
float m3 = M6;
float g = -9.81;

float c1 = cos(q1);
float s12 = sin(q1 + q2);
float s123 = sin(q1 + q2 + q3);

g123[0] = g * (m1 * r1 * c1 + m2 * (l1 * c1 + r2 * s12) +
    m3 * (l1 * c1 + l2 * s12 + r3 * s123));
g123[1] = g * (m2 * r2 * s12 + m3 * (l2 * s12 + r3 * s123));
g123[2] = g * (m3 * r3 * s123);
```

5.[12 Points] floatControl()

```
gv.tau = gv.G;
```

6.[12 Points] njgotoControl()

Results are shown in 3 and 4

7.[12 Points] jgotoControl()

The derivative component in the new PD controller reduces the oscillations caused by higher Kp gains. The gravity compensation also makes the work for the P controller

easier as it does not have to account for the constant force of gravity. The results are shown in Figures 5 and 6.

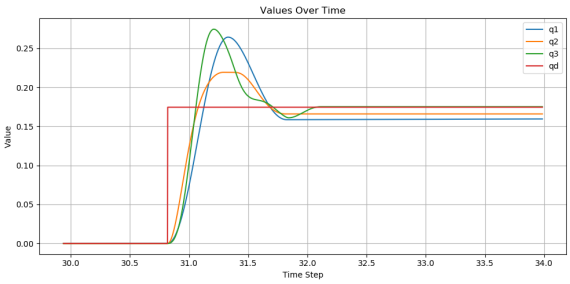

Figure 3: Desired joint states $qd*$ and actual joint states $q*$ for *njgotoControl* with Kp values: $q_1 = 400$, $q_2 = 200$, $q_3 = 25$.

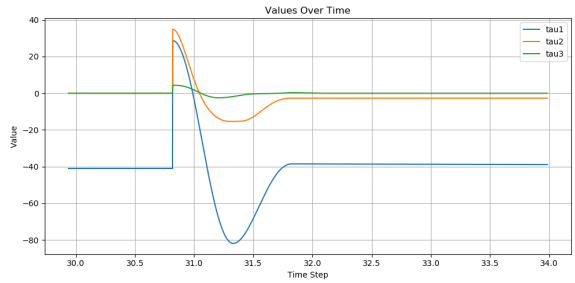

Figure 4: Applied torques $\tau*$ for *njgotoControl* with Kp values: $q_1 = 400$, $q_2 = 200$, $q_3 = 25$.

Contributions:

Student Name	**A**	**B1**	**B2**	**B3**	**B4**	**B5**	**B6**	**B7**
Yahuan Shi	x	x	x	x	x			x
Armin Puran Youssef	x					x	x	
Eckart Cobo-Briesewitz		x	x	x				x

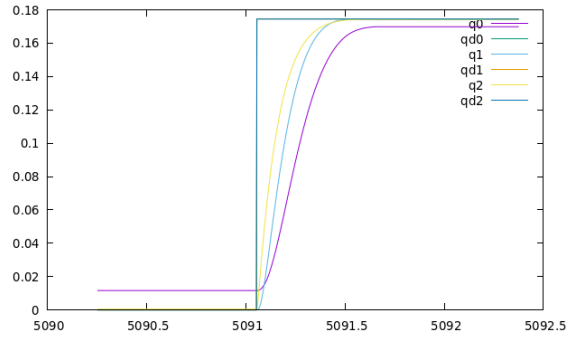

Figure 5: Desired joint states $qd*$ and actual joint states $q*$ for *jgotoControl* with Kp values: $q_1 = 600$, $q_2 = 400$, $q_3 = 400$, and Kv values: $q_1 = 100$, $q_2 = 40$, $q_3 = 40$.

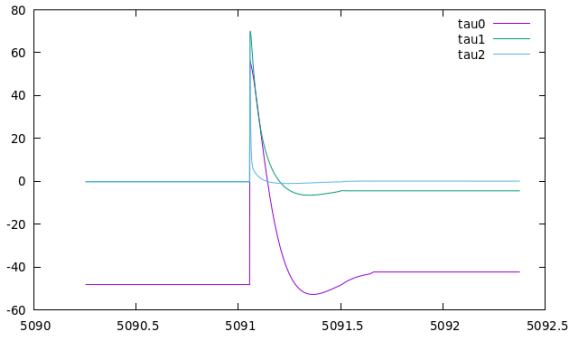

Figure 6: Applied torques $\tau*$ for *jgotoControl* with Kp values: $q_1 = 600$, $q_2 = 400$, $q_3 = 400$, and Kv values: $q_1 = 100$, $q_2 = 40$, $q_3 = 40$.